

This query makes a Cartesian product between the category table and the posts table. It can also be called a **cross join**:

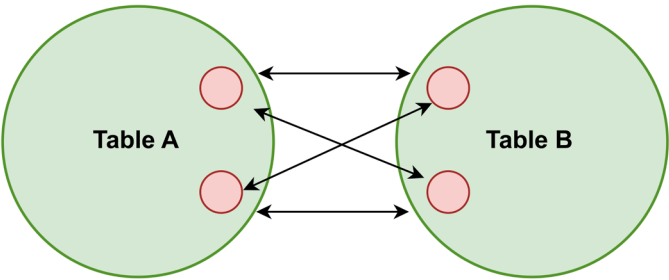


Figure 5.1: A cross join

The same query can also be written in the following way:

```
forumbd=>  select c.pk,c.title,p.pk,p.category,p.title from categories c
CROSS JOIN posts p;
pk |          title          | pk | category |          title
---+-----+-----+-----+-----
 1 | Database                |  1 |          1 | Indexing PostgreSQL
 2 | Unix                    |  1 |          1 | Indexing PostgreSQL
 3 | Programming Languages   |  1 |          1 | Indexing PostgreSQL
 4 | New Category            |  1 |          1 | Indexing PostgreSQL
 5 | Database                |  1 |          1 | Indexing PostgreSQL
 1 | Database                |  2 |          1 | Indexing Mysql
 2 | Unix                    |  2 |          1 | Indexing Mysql
 3 | Programming Languages   |  2 |          1 | Indexing Mysql
 4 | New Category            |  2 |          1 | Indexing Mysql
 5 | Database                |  2 |          1 | Indexing Mysql
 1 | Database                |  3 |          3 | Data types in C++
 2 | Unix                    |  3 |          3 | Data types in C++
 3 | Programming Languages   |  3 |          3 | Data types in C++
 4 | New Category            |  3 |          3 | Data types in C++
 5 | Database                |  3 |          3 | Data types in C++
(15 rows)
```

## Using INNER JOIN

Now suppose that starting with all the possible combinations that exist between the rows of the category table and the rows of the posts table, we want to filter all the rows that have the same value as the category field (`category.pk = posts.category`). We want to have a result like the one described in the following diagram:

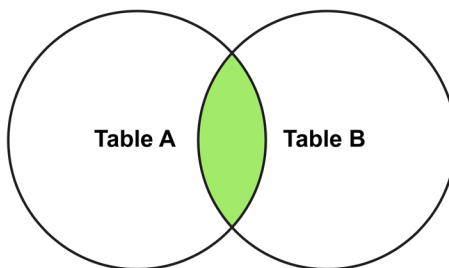


Figure 5.2: An inner join



The `INNER JOIN` keyword selects records that have matching values in both tables.

To achieve this, we need to run the following code:

```
forumbd=> select c.pk,c.title,p.pk,p.category,p.title from categories
c,posts p where c.pk=p.category;
pk | title | pk | category | title
----+-----+----+-----+-----
 1 | Database | 1 | 1 | Indexing PostgreSQL
 1 | Database | 2 | 1 | Indexing Mysql
 3 | Programming Languages | 3 | 3 | Data types in C++
(3 rows)
```

We can also write the same query using the explicit `JOIN` operation:

```
forumbd=> select c.pk,c.title,p.pk,p.category,p.title from categories c
inner join posts p on c.pk=p.category;
pk | title | pk | category | title
----+-----+----+-----+-----
 1 | Database | 1 | 1 | Indexing PostgreSQL
```

```

1 | Database          | 2 |          1 | Indexing Mysql
3 | Programming Languages | 3 |          3 | Data types in C++
(3 rows)

```

## INNER JOIN versus EXISTS/IN

If we wanted to search for all posts that belong to the Database category using the INNER JOIN condition, we would have to rewrite the query in this way:

```

forumbd=> \x
Expanded display is on.

forumbd=> select c.pk,c.title,p.pk,p.category,p.title from categories c
inner join posts p on c.pk=p.category where c.title='Database';
-[ RECORD 1 ]-----
pk          | 1
title       | Database
pk          | 1
category    | 1
title       | Indexing PostgreSQL
-[ RECORD 2 ]-----
pk          | 1
title       | Database
pk          | 2
category    | 1
title       | Indexing Mysql

```



Using the INNER JOIN condition, we can rewrite all queries that can be written using the IN or EXISTS condition.

It is preferable to use JOIN conditions whenever possible instead of IN or EXISTS conditions, because they perform better in terms of execution speed, as we will see in the following chapters.

## Using LEFT JOINS

We will now explore what a left join is. As an example, we can perform the following query:

```

forumdb=> select c.*,p.category,p.title from categories c left join posts
p on c.pk=p.category;
-[ RECORD 1 ]-----
pk          | 1
title       | Database
description | Database related discussions
category    | 1
title       | Indexing PostgreSQL
-[ RECORD 2 ]-----
pk          | 1
title       | Database
description | Database related discussions
category    | 1
title       | Indexing Mysql
-[ RECORD 3 ]-----
pk          | 3
title       | Programming Languages
description | All about programming languages
category    | 3
title       | Data types in C++
-[ RECORD 4 ]-----
pk          | 2
title       | Unix
description | Unix and Linux discussions
category    | (NULL)
title       | (NULL)
-[ RECORD 5 ]-----
pk          | 5
title       | Database
description | PostgreSQL
category    | (NULL)
title       | (NULL)
-[ RECORD 6 ]-----

```

```
pk      | 4
title   | New Category
description | (NULL)
category | (NULL)
title   | (NULL)
```

This query returns all records of the `categories` table and returns the matched records from the `posts` table. As we can see, if the second table (the `posts` table, in this example) has no matches, the result is `NULL`.



The `left join` keyword returns all records from the left table (`table1`), and all the records from the right table (`table2`). The result is `NULL` from the right side if there is no match.

This diagram gives us an idea of how a left join works:

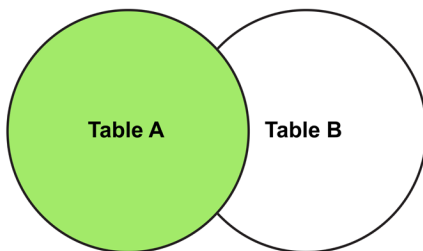


Figure 5.3: A left join

Suppose now that we want to search for all categories that do not have posts – we could write the following:

```
forumdb=> \x
Expanded display is off.

forumdb=> select * from categories c where c.pk not in (select category
from posts);
 pk | title      | description
----+-----+-----
  2 | Unix       | Unix and Linux discussions
  4 | New Category | (NULL)
  5 | Database   | PostgreSQL
(3 rows)
```

This query, written using the `NOT IN` condition, looks for all records in the `categories` table for which the `pk` value does not match in the `category` field of the `posts` table. As we have already seen, another way to write the same query would be to use the `NOT EXISTS` condition:

```
forumdb=> select * from categories c where not exists (select 1 from posts
where category=c.pk);
pk | title | description
---+-----+-----
2 | Unix | Unix and Linux discussions
4 | New Category | (NULL)
5 | Database | PostgreSQL
(3 rows)
```

If we now wanted to use a left join in order to achieve the same purpose, we would start by writing the following `left join` query:

```
forumdb=> \x
Expanded display is on.

forumdb=> select c.*,p.category from categories c left join posts p on
p.category=c.pk;
-[ RECORD 1 ]-----
pk          | 1
title       | Database
description | Database related discussions
category    | 1
-[ RECORD 2 ]-----
pk          | 1
title       | Database
description | Database related discussions
category    | 1
-[ RECORD 3 ]-----
pk          | 3
title       | Programming Languages
description | All about programming languages
category    | 3
-[ RECORD 4 ]-----
pk          | 2
```

```

title      | Unix
description | Unix and Linux discussions
category   | (NULL)
-[ RECORD 5 ]-----
pk         | 5
title      | Database
description | PostgreSQL
category   | (NULL)
-[ RECORD 6 ]-----
pk         | 4
title      | New Category
description | (NULL)
category   | (NULL)

```

From the result, it is immediately clear that all the values we are looking for are those for which the value of `p.category` is `NULL`.

So, we rewrite the query in the following way:

```

forumdb=> \x
Expanded display is off.

forumdb=> select c.* from categories c left join posts p on p.category=c.
pk where p.category is null;
 pk |   title   |      description
----+-----+-----
  2 | Unix      | Unix and Linux discussions
  4 | New Category | (NULL)
  5 | Database  | PostgreSQL
(3 rows)

```

As shown here, we get the same result we had using the `NOT EXISTS` or `NOT IN` condition.



Using the `left join` condition, we can rewrite some queries that can be written using the `IN` or `EXISTS` condition.

As mentioned earlier, it is preferable to use JOIN conditions whenever possible instead of IN or EXISTS conditions, because they perform better in terms of execution speed, as we will see in the following chapters.

## Using RIGHT JOIN

The right join is the twin of the left join, so we would have the same result if we wrote table A left join table B, or table B right join table A. For example, we can obtain the same results if we write:

```
select c.*,p.category from categories c left join posts p on p.category=c.pk;
```

or if we write:

```
select c.*,p.category,p.title from posts p right join categories c on c.pk=p.category;
```

as we can see here:

```
forumdb=> \x
Expanded display is on.

forumdb=> select c.*,p.category,p.title from posts p right join categories
c on c.pk=p.category;
-[ RECORD 1 ]-----
pk          | 1
title       | Database
description | Database related discussions
category    | 1
title       | Indexing PostgreSQL
-[ RECORD 2 ]-----
pk          | 1
title       | Database
description | Database related discussions
category    | 1
title       | Indexing Mysql
-[ RECORD 3 ]-----
pk          | 3
title       | Programming Languages
description | All about programming languages
```

```

category | 3
title    | Data types in C++
-[ RECORD 4 ]-----
pk       | 2
title    | Unix
description | Unix and Linux discussions
category | (NULL)
title    | (NULL)
-[ RECORD 5 ]-----
pk       | 5
title    | Database
description | PostgreSQL
category | (NULL)
title    | (NULL)
-[ RECORD 6 ]-----
pk       | 4
title    | New Category
description | (NULL)
category | (NULL)
title    | (NULL)

```



The **RIGHT JOIN** keyword returns all records from the right table (**table2**) and all records from the left table (**table1**) that match the right table (**table2**). The result is **NULL** from the left side when there is no match.

This diagram illustrates how **RIGHT JOIN** works:

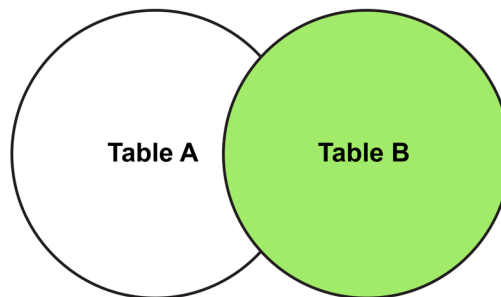


Figure 5.4: A right join

## Using FULL OUTER JOIN

In SQL, FULL OUTER JOIN is the combination of what we would have if we put together the right join and the left join. We will check it out using the following steps:

1. Let's create a new temporary table and insert some data:

```
forumbd=> create temp table new_posts as select * from posts;
SELECT 3
forumbd=> insert into new_posts (pk,title,content,author,category)
values (6,'A new Book','A new book not present in
categories....',1,NULL);
INSERT 0 1
```

2. Now, the current situation is as follows:

```
forumbd=> \x
Expanded display is off.
forumbd=> select pk,title,category from new_posts ;
```

| pk | title               | category |
|----|---------------------|----------|
| 1  | Indexing PostgreSQL | 1        |
| 2  | Indexing Mysql      | 1        |
| 3  | Data types in C++   | 3        |
| 6  | A new Book          | (NULL)   |

3. Now let's try to write this JOIN query:

```
forumbd=> select c.pk,c.title,p.pk,p.title from categories c inner
join new_posts p on p.category=c.pk;
```

| pk | title                 | pk | title               |
|----|-----------------------|----|---------------------|
| 1  | Database              | 1  | Indexing PostgreSQL |
| 1  | Database              | 2  | Indexing Mysql      |
| 3  | Programming Languages | 3  | Data types in C++   |

(3 rows)

This query returns all the records that have posts (in the table new\_post) and categories.